

Java Backend Architecture

Interview Series

Chapter 16.9

Docker Without Kubernetes

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

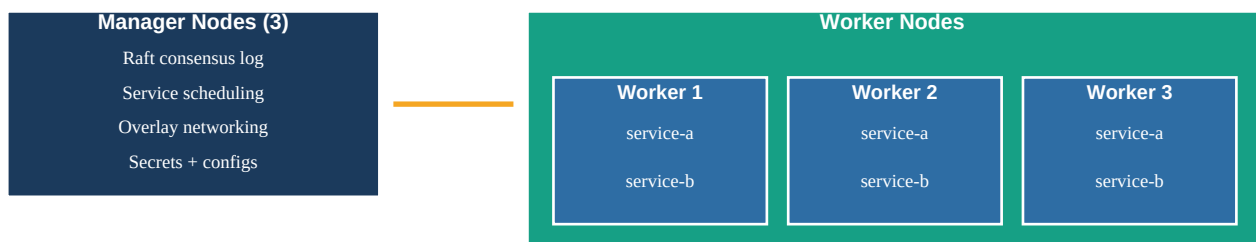
Chapter 16.9 – Docker Without Kubernetes

Container Orchestration Without Kubernetes

Kubernetes is not always the right choice. Docker Swarm, AWS ECS/Fargate, Azure Container Instances, and HashiCorp Nomad are production-grade container orchestration platforms with different complexity and capability trade-offs. Understanding these alternatives is essential for choosing the right tool for each use case.

Docker Swarm Architecture

Docker Swarm – Manager/Worker Architecture



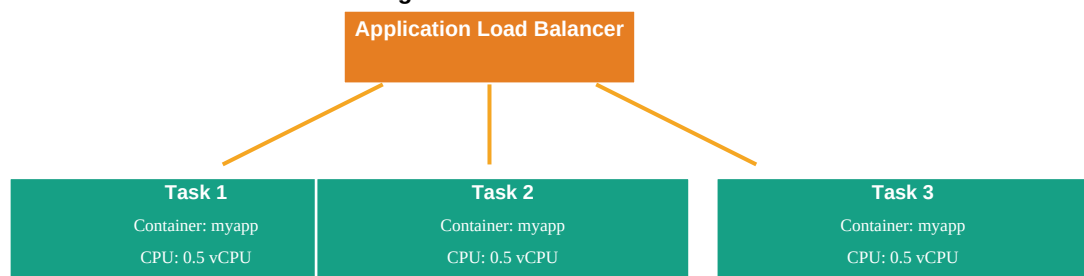
```
docker swarm init | docker service create --replicas 3 | --update-delay 10s rolling update
```

Docker Swarm Key Commands

```
# Initialize Swarm docker swarm init --advertise-addr 192.168.1.10 docker swarm join --token
<worker-token> 192.168.1.10:2377 # Create service with 3 replicas docker service create \
--name myapp \ --replicas 3 \ --publish 8080:8080 \ --network myoverlay \ --secret db_password
\ --update-delay 10s \ --update-parallelism 1 \ myapp:1.0 # Rolling update docker service
update --image myapp:1.1 myapp # Scale docker service scale myapp=5 # Status docker service ls
docker service ps myapp
```

AWS ECS + Fargate Architecture

AWS ECS Fargate – Task → Service → ALB



ECS Service (desired=3, min=2, max=10) + Auto Scaling + CloudWatch

Fargate: serverless compute — no EC2 to manage

ECS Task Definition (Java App)

```
{ "family": "myapp", "requiresCompatibilities": ["FARGATE"], "networkMode": "awsvpc", "cpu":
"512", "memory": "1024", "executionRoleArn": "arn:aws:iam::123:role/ecsTaskExecutionRole",
"taskRoleArn": "arn:aws:iam::123:role/myAppTaskRole", "containerDefinitions": [{ "name":
"myapp", "image": "123.dkr.ecr.us-east-1.amazonaws.com/myapp:1.0", "portMappings":
[{"containerPort": 8080}], "environment": [ {"name": "SPRING_PROFILES_ACTIVE", "value":
```

```
"prod"} ], "secrets": [{ "name": "DB_PASSWORD", "valueFrom":
"arn:aws:secretsmanager:us-east-1:123:secret:db-password" }], "logConfiguration": {
"logDriver": "awslogs", "options": {"awslogs-group": "/ecs/myapp", "awslogs-region":
"us-east-1"} }, "healthCheck": { "command": ["CMD-SHELL", "curl -f
http://localhost:8080/actuator/health || exit 1"], "interval": 30, "timeout": 10, "retries": 3,
"startPeriod": 60 } } ] }
```

ECS Service Autoscaling

```
# ECS Service autoscaling using Application Auto Scaling aws application-autoscaling
register-scalable-target \ --service-namespace ecs \ --resource-id service/myCluster/myapp \
--scalable-dimension ecs:service:DesiredCount \ --min-capacity 2 --max-capacity 10 # Scale on
CPU utilization aws application-autoscaling put-scaling-policy \ --policy-name
myapp-cpu-scaling \ --service-namespace ecs \ --resource-id service/myCluster/myapp \
--scalable-dimension ecs:service:DesiredCount \ --policy-type TargetTrackingScaling \
--target-tracking-scaling-policy-configuration \ TargetValue=70.0,PredefinedMetricSpecificatio
n={PredefinedMetricType=ECSServiceAverageCPUUtilization}
```

Swarm vs ECS vs Kubernetes Decision Matrix

Swarm vs ECS vs Kubernetes – Decision Matrix

Docker Swarm	AWS ECS/Fargate	Kubernetes
Simple to set up	AWS-managed	Most flexible
docker CLI native	Fargate: serverless	Rich ecosystem
Swarm secrets	IAM integration	Steeper learning
No autoscaling	ALB/CloudMap	Self-hosted or managed
Limited ecosystem	AWS lock-in	Industry standard

Factor	Docker Swarm	AWS ECS/Fargate	Kubernetes
Setup complexity	Low	Low (managed)	High
Learning curve	Low	Medium	High
Auto-scaling	Manual only	ASG / ECS Auto Scaling	HPA / VPA / KEDA
Service discovery	DNS (VIP)	CloudMap / ALB	kube-dns / Service
Secrets	Swarm secrets (Raft)	Secrets Manager / SSM	K8s Secrets / Vault
Rolling updates	Built-in	ECS rolling deploy	Deployment strategy
Multi-cloud	Yes	AWS only	Yes
Best for	Small teams, simple apps	AWS-native workloads	Complex, large-scale

Fargate Spot

Fargate Spot uses spare AWS capacity at up to 70% discount vs. standard Fargate. Spot tasks can be interrupted with 2-minute notice. Design for interruption:

```
# ECS Service with mixed Spot + On-Demand aws ecs create-service \ --cluster myCluster \
--service-name myapp \ --task-definition myapp:5 \ --desired-count 6 \
--capacity-provider-strategy \ capacityProvider=FARGATE,weight=1,base=2 \
capacityProvider=FARGATE_SPOT,weight=3 \ --load-balancers
targetGroupArn=arn:...,containerName=myapp,containerPort=8080
```

Interview Q&A; – Docker Without Kubernetes

Q1. What is Docker Swarm and how does it differ from Kubernetes?

Docker Swarm is Docker's native clustering and orchestration tool. Manager nodes maintain cluster state using the Raft consensus algorithm. Services define desired state (image, replicas, ports). Swarm schedules containers on worker nodes. Key differences from Kubernetes: simpler setup (docker swarm init), uses standard docker CLI (no separate kubectl), built-in overlay networking, Swarm secrets. Drawbacks vs K8s: no horizontal pod autoscaling, limited ecosystem (no Helm equivalent), less feature-rich scheduling. Best for: small teams needing orchestration without Kubernetes complexity.

Q2. How does Docker Swarm implement rolling updates?

docker service update --image myapp:1.1 myapp performs a rolling update. Configure with: --update-delay (wait time between updating task groups), --update-parallelism (how many tasks to update at once), --update-failure-action (pause/continue/rollback on failure), --rollback-delay. Swarm stops old tasks and starts new ones in batches. If healthchecks fail, --update-failure-action rollback reverts automatically. Monitor with docker service ps myapp. To rollback manually: docker service rollback myapp.

Q3. What is AWS ECS and how does it work?

ECS (Elastic Container Service) is AWS's managed container orchestration service. Key concepts: Task Definition (like a Dockerfile for a multi-container setup — defines containers, CPU/memory, IAM roles, networking, logging), Task (running instance of a task definition), Service (manages desired count of tasks, integrates with ALB, handles rolling deployments), Cluster (logical grouping of infrastructure). Launch types: EC2 (you manage the instances) or Fargate (AWS manages compute). ECS integrates natively with ALB, CloudWatch, IAM, Secrets Manager, and Service Discovery.

Q4. What is the difference between AWS ECS EC2 launch type and Fargate?

EC2 launch type: you provision and manage EC2 instances in the cluster. You pay for the instances (whether tasks run or not). You choose instance types, manage OS patching, and scale the cluster. Fargate launch type: AWS manages the compute infrastructure. You pay only for the exact CPU/memory allocated to running tasks. No instances to manage, patch, or scale. Fargate is serverless — tasks start immediately without waiting for instance capacity. Use Fargate when: you want operational simplicity and don't need GPU or custom instance types. Use EC2 when: cost optimization is critical (reserved instances) or you need GPU workloads.

Q5. How does ECS service discovery work?

ECS supports two service discovery mechanisms: (1) AWS Cloud Map: registers each task in a private DNS namespace (myapp.myservice.local). Applications resolve service names via Route53 private hosted zone. Supports both DNS (A records) and API-based discovery. (2) ALB-based: ECS registers tasks as ALB target group targets; ALB routes by path/host-based rules. For internal service-to-service: use Cloud Map for direct container-to-container communication. For external traffic: use ALB with target groups. ECS also supports App Mesh for service mesh-based discovery.

Q6. What are ECS IAM task roles and how do they differ from execution roles?

ECS has two IAM role concepts: (1) Task Execution Role (ecsTaskExecutionRole): used by the ECS agent to pull images from ECR, write CloudWatch logs, read Secrets Manager/SSM for injecting environment variables. You don't write code to use this — ECS uses it automatically. (2) Task Role (myAppTaskRole): used BY your application code at runtime. When your Java app calls AWS SDK (S3, DynamoDB, SQS), it uses the task role credentials via the ECS metadata endpoint. This is the ECS equivalent of EC2 instance role / Kubernetes IRSA.

Q7. When would you choose Docker Swarm over Kubernetes?

Choose Swarm when: (1) Small team with limited K8s expertise and no time to learn it. (2) Simple applications without complex scheduling requirements. (3) Already using Docker Compose locally and want similar semantics in production (docker stack deploy uses Compose file format). (4) Limited budget — no managed K8s costs. (5) Self-hosted environment without K8s infrastructure team. Choose Kubernetes when: you need autoscaling, rich ecosystem (Helm, Istio, Argo CD, KEDA), multi-cloud portability, fine-grained resource management, or you have a team that knows K8s.

Q8. What is HashiCorp Nomad and how does it compare to ECS/Kubernetes?

Nomad is a workload orchestrator that supports containers (Docker), VMs, Java JARs, raw executables, and system services in a single scheduler. Key advantages: (1) Simpler than K8s — single binary, minimal dependencies. (2) Multi-workload: schedule containers + non-containerized workloads together. (3) Multi-cloud and on-premises native. (4) Integrates with Consul (service discovery) and Vault (secrets). (5) Federation across datacenters. Use Nomad when you need to orchestrate diverse workloads beyond just containers, or want Kubernetes power with less complexity.

Q9. What is Azure Container Instances (ACI) and when is it useful?

ACI provides on-demand, serverless container execution without managing VMs or clusters. Start a container in seconds: `az container create --image myapp:latest --cpu 1 --memory 1.5`. No persistent cluster needed — pay only for running time. Use cases: (1) Batch jobs and event-driven processing. (2) CI/CD build agents (start on demand, terminate after job). (3) Quick testing/demos without cluster setup. (4) Burst capacity from AKS via Virtual Nodes (ACI as overflow capacity). Limitations: no persistent orchestration, no built-in service discovery, limited networking.

Q10. How does Swarm handle secrets and how do they compare to Kubernetes Secrets?

Docker Swarm secrets are stored encrypted in the Raft log (AES-256-GCM), distributed to workers over TLS, and mounted as tmpfs files at `/run/secrets/name`. Never stored on disk on workers. Kubernetes Secrets are base64-encoded (NOT encrypted by default) stored in etcd. K8s secrets require encryption-at-rest configuration (EncryptionConfiguration) for security. With RBAC, K8s Secrets can be locked down. Both support external secret backends via operators (K8s External Secrets Operator, Vault Agent). For Swarm: secrets are more secure by default. For K8s: more flexible with proper configuration.

Q11. What is ECS service autoscaling and how do you configure it?

ECS services scale using AWS Application Auto Scaling. Target tracking policy: maintain a metric at a target value (e.g. CPU = 70%). Step scaling: scale by specific amounts when metric crosses thresholds. Scheduled scaling: scale on a cron schedule (e.g. scale up before business hours). Metrics: `ECSServiceAverageCPUUtilization`, `ECSServiceAverageMemoryUtilization`, `ALBRequestCountPerTarget` (from ALB). Setup: register scalable target (ECS service), create scaling policy. Fargate scales instantly (no instance warm-up). EC2 requires cluster capacity provider to add instances first.

Q12. What are the advantages of Fargate over managing EC2 clusters?

Fargate advantages: (1) No cluster management — no EC2 patching, no instance type decisions, no AMI updates. (2) Right-sizing per task — define exact CPU/memory; pay only for what you need. (3) Faster scaling — no instance launch delay. (4) Security — each task gets its own kernel + network namespace (stronger isolation than EC2 tasks). (5) No over-provisioning — no idle EC2 instances. (6) Fargate Spot for 70% savings. Trade-offs: slightly higher cost than reserved EC2 at high, stable utilization; no GPU support; task start time slightly slower than EC2 (new kernel allocation).

Q13. How does ECS handle rolling deployments?

ECS rolling deployment: configure in service: `minimumHealthyPercent` (min % of running tasks during deploy) and `maximumPercent` (max % of tasks including new). Default: min 100%, max 200% (new tasks start before old ones stop — zero downtime). With ALB health checks: ECS waits for new tasks to pass health checks before draining old tasks. `healthCheckGracePeriodSeconds` gives JVM time to start before health checks begin. Blue-green deployments: use CodeDeploy integration with ECS — deploy new task set, run validation, shift traffic, optionally rollback.

Q14. What is ECS CloudWatch logging and how do you configure it?

ECS tasks write logs to CloudWatch using the `awslogs` log driver. Configure in task definition `logConfiguration`: `logDriver=awslogs`, `options`: `awslogs-group (/ecs/myapp)`, `awslogs-region (us-east-1)`, `awslogs-stream-prefix (ecs)`. The task execution role needs `logs:CreateLogGroup`, `logs:CreateLogStream`, `logs:PutLogEvents` permissions. Logs are queryable with CloudWatch Logs Insights. For structured logging: use JSON format (Spring Boot: `spring-boot-starter-logging` with `logstash-logback-encoder`) — CloudWatch can filter on JSON fields. Alternative: send logs to OpenSearch/Datadog via Firelens (Fluent Bit sidecar).

Q15. When should you NOT use Kubernetes?

Avoid Kubernetes when: (1) Your team is small (< 5 engineers) and has no K8s experience — operational overhead is significant. (2) Your application is a simple monolith or few services. (3) Time-to-production is critical — K8s setup takes weeks to do properly. (4) You're on AWS with existing ECS expertise — ECS is operationally simpler for AWS-native deployments. (5) Cost is paramount for small workloads — K8s control plane costs (\$150+/month on EKS) don't make sense for a single service. (6) Compliance requirements limit managed K8s. Use Docker Compose (dev), Swarm (simple), or ECS/Fargate (AWS) as alternatives.

Q16. What is the Nomad task driver system and what workloads can it run?

Nomad uses task drivers to execute different workload types: `docker` (Docker containers via `containerd`), `podman` (rootless container runtime), `exec` (raw binary execution on host), `java` (run JAR files directly without containers), `qemu` (VMs), `raw_exec` (privileged host execution), `rkt` (deprecated). Each driver has its own resource limits, networking, and security model. A single Nomad cluster can run containerized microservices alongside legacy Java JARs, batch scripts, and system daemons. This polyglot capability makes Nomad attractive for organizations with diverse workload types during containerization migration.